AIMLF1 · PROYECTO FINAL DEL BOOTCAMP · EN PAREJAS

Olist: del pedido a la reseña

Un proyecto de punta a punta con datos reales: SQL, un modelo de riesgo evaluado con meses que nunca verán, una pregunta abierta, un modelo de deep learning para leer comentarios, una API y una app publicadas, y una ficha del modelo que defenderán en vivo.

Modalidad: parejas · Valor: 100 puntos (+5 extra) · Entrega final: [completar]

Contenido

1. El problema
2. Qué van a construir
3. Los datos
4. El contrato
5. Parte 1 · Datos
6. Parte 2 · Modelo de riesgo
7. Parte 3 · Pregunta abierta
8. Parte 4 · Deep learning
9. Extra · Red neuronal contra XGBoost
10. Parte 5 · Despliegue
11. Parte 6 · Ficha del modelo y defensa
12. Rúbrica
13. Hitos, entrega y reglas
14. Preguntas frecuentes

1. El problema

Olist es una plataforma brasileña que conecta pequeñas tiendas con los grandes sitios de comercio electrónico. Cuando un pedido llega, el cliente recibe una encuesta y califica su compra de 1 a 5 estrellas, con un comentario opcional. Una reseña de 1 o 2 estrellas daña la reputación del vendedor y de la plataforma.

El equipo de atención al cliente puede llamar a algunos clientes justo después de la entrega para resolver problemas **antes** de que escriban la reseña. No puede llamar a todos: su capacidad alcanza para **1 de cada 10 pedidos**.

Las preguntas del proyecto:

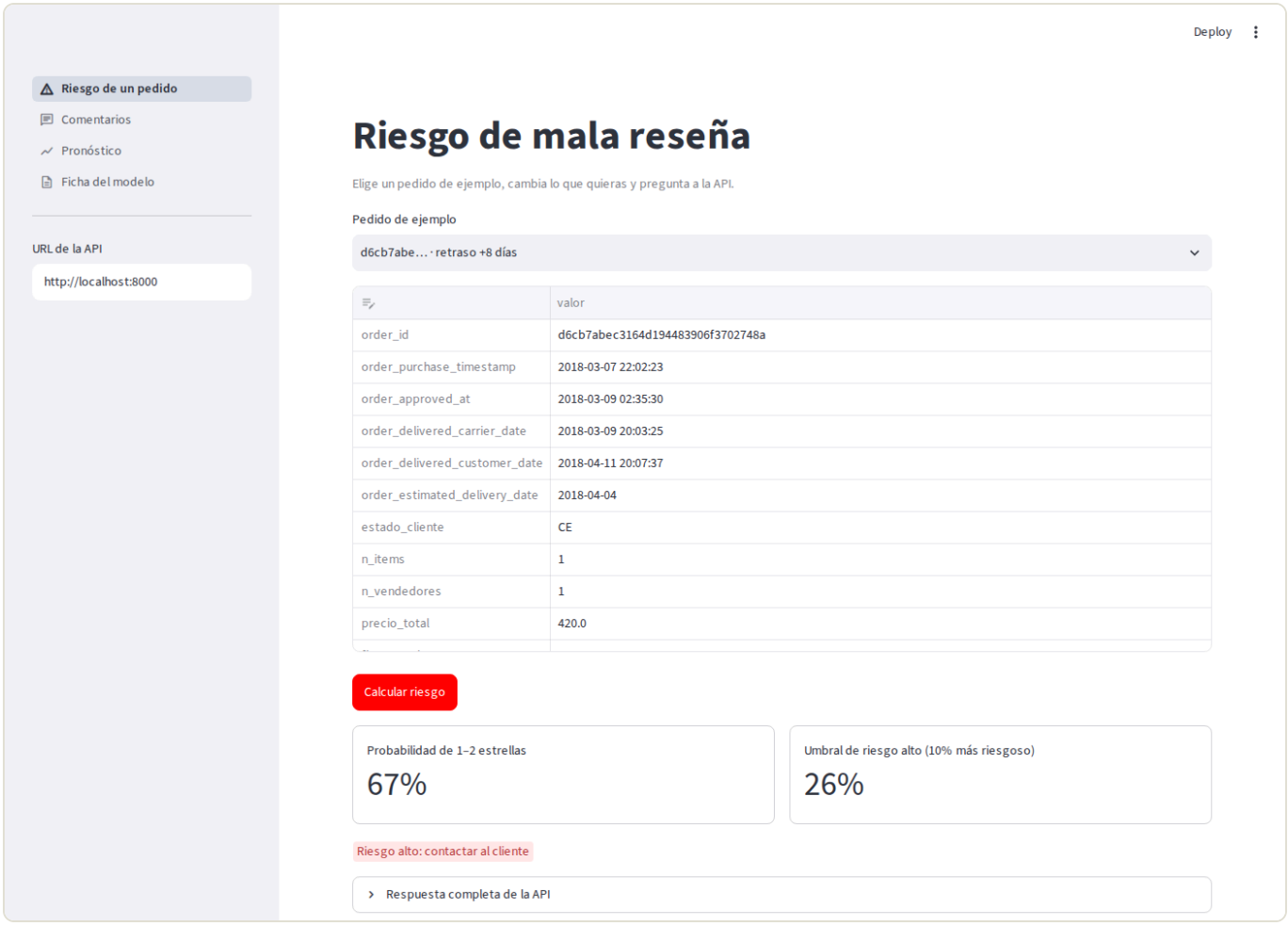
- En el momento en que un pedido se entrega, ¿qué tan probable es que reciba una mala reseña? ¿A quiénes llamar primero?
- Cuando los clientes escriben, ¿qué dicen? ¿Coincide lo que escriben con las estrellas que ponen?
- Y una pregunta que ustedes eligen: ¿cuántos pedidos llegarán el próximo mes, o qué tipos de vendedores hay?

Es el cierre del bootcamp: usarán lo de los tres módulos. SQL y limpieza del Módulo 1; pipelines, clasificación, XGBoost, validación, K-Means, series de tiempo, FastAPI y Streamlit del Módulo 2; y modelos preentrenados de las dos semanas de deep learning.

2. Qué van a construir

Parte	Qué entregan	Puntos
1. Datos	Una consulta SQL que une al menos 4 tablas y produce la tabla del contrato; limpieza y diccionario de variables	15
2. Modelo de riesgo	Un pipeline que predice la probabilidad de mala reseña, guardado como `modelo_riesgo.joblib`. Se evalúa con pedidos que ustedes no tienen	25
3. Pregunta abierta	A elegir: pronóstico de pedidos diarios de junio de 2018, o segmentación de vendedores	10
4. Deep learning	Sentimiento de los comentarios con un modelo preentrenado, comparado con las estrellas	15
5. Despliegue	API en Render y app de Streamlit publicada que la consume	15
6. Ficha y defensa	Ficha del modelo y defensa en vivo de 8 minutos	20
Extra	Red neuronal de Keras contra su XGBoost	+5

Así se ve la página principal de una app de referencia: se elige un pedido, la app consulta la API y muestra el riesgo. No tienen que copiar el diseño.



App de referencia. La probabilidad y el umbral vienen de la API desplegada.

3. Los datos

Son los datos públicos de Olist: unos 100,000 pedidos reales entre 2016 y 2018 (repositorio oficial `github.com/olist/work-at-olist-data`, licencia MIT). Están cargados en la base de datos del curso, en el esquema `final`, con el mismo usuario de solo lectura de las semanas 5 y 6.

Tabla	Una fila por	Columnas clave
`final.orders`	pedido	`order_id`, `customer_id`, `order_status`, las 5 fechas del pedido
`final.order_items`	artículo de un pedido	`order_id`, `order_item_id` (1, 2, 3…), `product_id`, `seller_id`, `price`, `freight_value` (flete)
`final.order_payments`	pago de un pedido	`order_id`, `payment_sequential`, `payment_type`, `payment_installments` (cuotas), `payment_value`
`final.order_reviews`	reseña	`order_id`, `review_score` (1 a 5), `review_comment_message`, `review_creation_date`, `review_answer_timestamp`
`final.customers`	cliente de un pedido	`customer_id`, `customer_unique_id`, `customer_state`
`final.products`	producto	`product_id`, `product_category_name`, `product_weight_g`, `product_photos_qty` (los nombres traen errores de origen, como `lenght`)
`final.sellers`	vendedor	`seller_id`, `seller_state`
`final.category_translation`	categoría	nombre en portugués e inglés

> **Faltan meses a propósito.** Los pedidos comprados desde el **1 de junio de 2018** no están en `final`: los tiene el docente y con ellos se evalúan sus modelos. Es lo que pasa en la vida real: el modelo se entrena con el pasado y se usa con pedidos que todavía no existen.

Los textos de los comentarios están en **portugués**. Los estados son siglas de Brasil (SP = São Paulo, RJ = Río de Janeiro…).

4. El contrato

Un **contrato** es un acuerdo fijo sobre qué columnas entran a un sistema y qué significa cada una. Su modelo, su API y el calificador del docente usan el mismo contrato: si su modelo necesita una columna que no está aquí, **no correrá** con los pedidos ocultos.

La tabla del contrato tiene **una fila por pedido entregado y con reseña** (`order_status = 'delivered'` y fecha de entrega conocida), con estas 18 columnas, en este orden:

Columna	Definición exacta
`order_id`	identificador del pedido
`order_purchase_timestamp`, `order_approved_at`, `order_delivered_carrier_date`, `order_delivered_customer_date`, `order_estimated_delivery_date`	las cinco fechas del pedido, tal como vienen en `orders`: compra, aprobación del pago, entrega al transportista, entrega al cliente y fecha prometida
`estado_cliente`	`customer_state` del cliente del pedido
`n_items`	número de filas del pedido en `order_items`
`n_vendedores`	número de vendedores distintos del pedido
`precio_total`, `flete_total`	suma de `price` y de `freight_value` de los artículos
`peso_total_g`	suma del peso de los productos de los artículos
`fotos_promedio`	promedio de `product_photos_qty` de los artículos
`categoria`, `estado_vendedor`	categoría del producto y estado del vendedor del **artículo 1** (`order_item_id = 1`)
`tipo_pago`	`payment_type` del pago de **mayor monto** del pedido
`cuotas_max`	máximo de `payment_installments` del pedido
`mala_resena`	1 si `review_score` es 1 o 2; 0 si no. Si un pedido tiene varias reseñas, se usa la de `review_answer_timestamp` más reciente

Con los datos de `final` la tabla tiene **77,304 pedidos**. Si su consulta da otro número, revisen las reglas antes de seguir.

> **Pista de SQL (PostgreSQL):** conviene armar la consulta por partes con `WITH`: una subconsulta que agregue los artículos por pedido, otra para el artículo 1, otra para los pagos y otra para la reseña, y al final unirlas con `orders` y `customers`. Para quedarse con una sola reseña por pedido sirve `SELECT DISTINCT ON (order_id) ... ORDER BY order_id, review_answer_timestamp DESC`.

5. Parte 1 · Datos (15 puntos)

- Su consulta SQL, escrita por ustedes, que produce la tabla del contrato. La celda de la plantilla que verifica el contrato debe imprimir `True`.
- En Markdown: qué decidieron con pedidos de varios artículos, varios pagos y varias reseñas, y por qué.
- Revisión de faltantes y valores raros, con lo que hicieron con cada uno.
- Un **diccionario de variables**: cada variable que su modelo calcula a partir del contrato, cómo se calcula y qué significa para el negocio.

6. Parte 2 · Modelo de riesgo (25 puntos)

Qué se predice y cuándo

La probabilidad de que `mala_resena = 1`, calculada **en el momento en que el pedido se entrega**. En ese momento se conocen las fechas del pedido (incluida la entrega real), los artículos, los pagos y la geografía. **No se conoce nada de la reseña.**

> **Fuga de datos.** Usar el texto, el largo del comentario o las fechas de la reseña como variables hace que el modelo se vea excelente en su notebook, pero es trampa: esa información aparece *después* de la predicción. Además, esas columnas no están en el contrato, así que el modelo no correrá con los pedidos ocultos y la parte de desempeño valdrá 0.

La métrica: precisión en el 10% de mayor riesgo

Ordenen los pedidos de mayor a menor probabilidad, tomen el 10% de arriba (los que atención al cliente llamaría) y calculen qué proporción de ellos sí tuvo mala reseña.

Ejemplo: si en 1,000 pedidos el modelo marca los 100 más riesgosos y 40 de ellos terminaron con 1 o 2 estrellas, la precisión en el 10% es **0.40**. Si se eligieran 100 pedidos al azar, saldría cerca de la tasa general de malas reseñas.

```
k = int(len(y) * 0.10)
orden = np.argsort(-probabilidades)       # de mayor a menor riesgo
precision_10 = y.values[orden[:k]].mean()
```

Lo que se pide

- **Validación por fecha:** entrenen con los pedidos más antiguos y validen con los más recientes. Nada de `train_test_split` al azar para la evaluación principal.
- **Dos puntos de comparación:** el azar (la tasa de malas reseñas del periodo de validación) y la **regla del retraso**: ordenar los pedidos por días de retraso contra la fecha prometida, sin modelo.
- Al menos dos modelos distintos (por ejemplo, regresión logística y XGBoost), con un pipeline de scikit-learn.
- Las funciones que convierten las columnas del contrato en variables van en un archivo `preparacion.py`, que el notebook importa. El pipeline guardado las necesita al cargarse.
- El modelo final, entrenado con todos los pedidos de `final`, guardado como `modelo_riesgo.joblib`, y la prueba de la plantilla: cargado desde el archivo, predice con filas que solo tienen las columnas del contrato.
- Respuestas escritas, con cifras de sus salidas: cuánto mejoran sobre la regla y el azar, qué variables pesan más y por qué tiene sentido, y qué pasaría con el modelo en un periodo con muy pocos retrasos.

Cómo se califica el desempeño (10 de los 25 puntos)

El docente carga su `modelo_riesgo.joblib` con su `preparacion.py` y calcula la precisión en el 10% con los pedidos de **junio a agosto de 2018**.

Precisión en el 10% con los pedidos ocultos	Puntos
0.32 o más	10
0.28 a 0.32	8
0.24 a 0.28	6
Supera a la regla del retraso	4
Corre, pero no supera a la regla del retraso	2
No carga, o no corre con las columnas del contrato	0

No esperen ver en junio–agosto la misma cifra que en su validación: el periodo oculto es distinto, y parte del proyecto es pensar por qué.

7. Parte 3 · Pregunta abierta (10 puntos)

Elijan **una** de las dos opciones y justifiquen la elección.

Opción 3A · Pronóstico de pedidos diarios de junio de 2018

- Serie: pedidos creados por día (todas las filas de `orders`, cualquier estado) hasta el 31 de mayo de 2018.
- Pronóstico para cada día del **1 al 30 de junio de 2018** con un intervalo de 80%.
- Comparación con un pronóstico ingenuo (por ejemplo, repetir la última semana) usando validación por fecha.
- Archivo `pronostico_junio.csv` con columnas `fecha, pronostico, bajo, alto` (30 filas).
- Revisen con cuidado las últimas semanas de mayo antes de decidir.

El docente compara su archivo con los pedidos reales de junio: 2 puntos si el error medio absoluto es menor que el del ingenuo semanal, 1 más si es de 45 pedidos o menos, y 1 más si el intervalo cubre entre 65% y 95% de los días. Los otros 6 puntos son el método y la interpretación.

Opción 3B · Segmentación de vendedores

- Una fila por vendedor con al menos 10 pedidos entregados, y variables que ustedes definen con SQL (ventas, pedidos, flete relativo, tasa de retraso, categorías…).
- Preparación para K-Means (escalado; logaritmo si hay colas largas) y elección de k con codo y silueta, más el criterio de negocio.
- Perfil de cada grupo, nombre por su perfil y una acción concreta para cada segmento.
- Una visualización de los grupos (por ejemplo, con PCA).

8. Parte 4 · Deep learning: sentimiento de los comentarios (15 puntos)

Un **modelo preentrenado** es una red neuronal que otros ya entrenaron con millones de textos; ustedes solo la usan. Hugging Face es el sitio donde se publican miles de estos modelos, y la librería `transformers` los descarga y los usa con una línea.

Modelo sugerido: `cardiffnlp/twitter-xlm-roberta-base-sentiment`, entrenado en 8 idiomas, **portugués incluido**. Devuelve `Positive`, `Neutral` o `Negative`. Pueden usar otro si entiende portugués y lo justifican.

```
!pip install -q transformers
from transformers import pipeline
clasificador = pipeline("sentiment-analysis",
                        model="cardiffnlp/twitter-xlm-roberta-base-sentiment", device=0)
# device=0: GPU
clasificador(["Produto chegou antes do prazo, recomendo!"])
```

> Activen la GPU de Colab: *Entorno de ejecución › Cambiar tipo de entorno de ejecución › T4 GPU*. Sin GPU, quiten `device=0`; funciona, pero más lento. El modelo pesa alrededor de 1 GB: la primera descarga tarda.

- Una muestra de **al menos 1,000 comentarios** con texto (`review_comment_message` no vacío), con semilla fija.
- Las estrellas agrupadas igual que el modelo: 1–2 negativo, 3 neutro, 4–5 positivo. Tabla cruzada estrellas contra sentimiento y porcentaje de coincidencia.
- Al menos **5 desacuerdos** leídos y clasificados: ¿se equivocó el modelo (ironía, textos muy cortos) o el cliente (puso 5 estrellas y se quejó)?
- Conclusión: qué dice el texto que las estrellas no dicen, y para qué le serviría a Olist.
- Guarden los resultados en un CSV: la app los muestra.

Esta parte usa los comentarios como **tema de análisis**, no como variable del modelo de riesgo de la Parte 2: ahí siguen prohibidos.

9. Extra · Red neuronal contra XGBoost (+5 puntos)

Entrenen una red densa de Keras con las mismas variables y la misma división por fecha de la Parte 2. Muestren la curva de pérdida por época, usen *early stopping* (detener el entrenamiento cuando la validación deja de mejorar) y comparen con su XGBoost. Expliquen el resultado: en datos de tabla, ¿por qué gana uno u otro?

10. Parte 5 · Despliegue (15 puntos)

La API (Render)

Una API (interfaz de programación) es un programa que recibe datos por internet y responde con un resultado, como en la clase de FastAPI. La suya recibe un pedido y devuelve su riesgo.

Método y ruta	Recibe	Devuelve
`GET /`	nada	un JSON que confirma que está viva
`POST /riesgo`	un JSON con las 17 columnas del contrato (todas menos `mala_resena`); las fechas como texto `"2018-03-07 22:02:23"`; las que pueden faltar, opcionales	`{"probabilidad": 0.67, "riesgo_alto": true, ...}`

- Validación con Pydantic: un precio negativo o un campo obligatorio faltante responde 422.
- `riesgo_alto` es `true` si la probabilidad alcanza el umbral del 10% de mayor riesgo, calculado con sus datos de entrenamiento.
- En la carpeta de la API: `main.py`, `preparacion.py`, `modelo_riesgo.joblib`, el archivo del umbral y `requirements.txt`.
- El `requirements.txt` debe fijar **las mismas versiones** de pandas, numpy, scikit-learn y xgboost de su Colab (la plantilla las imprime). Con otras versiones el `.joblib` puede no cargar.
- Render: *Build command* `pip install -r requirements.txt`; *Start command* `uvicorn main:app --host 0.0.0.0 --port $PORT`.

La app (Streamlit Community Cloud)

- **Riesgo de un pedido:** el usuario elige o edita un pedido, la app llama a `POST /riesgo` de su API y muestra la probabilidad y si es riesgo alto.
- **Comentarios:** los resultados de la Parte 4, con la tabla cruzada y los desacuerdos para explorar.
- **Pregunta abierta:** el pronóstico con su intervalo, o los segmentos con sus perfiles.
- **Ficha del modelo** (Parte 6).
- Usen varias páginas (`st.navigation`) y la URL de la API en `st.secrets`, como en la actividad de Taxis NYC.

> Render apaga los servicios gratuitos que no reciben visitas: la primera consulta puede tardar hasta un minuto. Su app debe mostrar un mensaje claro si la API no responde, no un error de Python.

11. Parte 6 · Ficha del modelo y defensa (20 puntos)

Ficha del modelo (`ficha_modelo.md`)

Un documento corto para alguien que **no** sabe de machine learning y va a decidir si usa el modelo. Debe responder:

1. **Para qué sirve** y para qué **no** debe usarse.
2. **Datos:** de dónde, qué periodo, cuántos pedidos, qué se excluyó y por qué.
3. **Qué tan bien funciona:** su validación por fecha contra el azar y la regla, en palabras simples ("de cada 10 clientes que llamamos, unos X iban a...").
4. **Límites conocidos:** cuándo puede fallar.
5. **A quién afecta un error:** qué pasa con un falso positivo, con un falso negativo y si alguien puede salir perjudicado por un mal uso (por ejemplo, vendedores castigados por retrasos que no controlan).
6. **Recomendación:** cómo usarlo, con qué supervisión humana y cada cuánto revisarlo.

Defensa (8 minutos por pareja)

- 3 minutos: demostración de la app en vivo, con la API desplegada.
- 5 minutos: preguntas del docente sobre **su propio código y sus decisiones**. Cualquier integrante puede responder sobre cualquier parte, así que los dos deben entender todo el proyecto.

12. Rúbrica (100 puntos + 5 extra)

Parte	Criterio	Pts
1. Datos (15)	Consulta SQL propia que une al menos 4 tablas y cumple el contrato (la verificación imprime `True`)	6
	Decisiones justificadas: varios artículos, pagos y reseñas por pedido	4
	Revisión de faltantes y valores raros, y diccionario de variables	5
2. Modelo de riesgo (25)	Desempeño con los pedidos ocultos (escala de la sección 6)	10
	Validación por fecha, comparación con el azar y con la regla del retraso, al menos dos modelos	6
	Sin fuga de datos; `preparacion.py` y `.joblib` que cargan y corren con el contrato	4
	Respuestas escritas con cifras de sus salidas	5
3. Pregunta abierta (10)	3A: resultado contra los datos reales de junio (4) y método e interpretación del intervalo y de los supuestos (6)	10
	3B: preparación y elección de k (4), perfiles y nombres (3), acción por segmento y visualización (3)	(10)
4. Deep learning (15)	Modelo preentrenado aplicado a al menos 1,000 comentarios	5
	Tabla cruzada con las estrellas y porcentaje de coincidencia	4
	Al menos 5 desacuerdos leídos y clasificados	4
	Conclusión: qué agrega el texto a las estrellas	2
5. Despliegue (15)	La API responde en Render (`GET /` y `POST /riesgo`)	4
	Validación con Pydantic y documentación en `/docs`	2
	App publicada con las páginas de riesgo (vía API), comentarios y pregunta abierta	6
	Ficha visible en la app, mensaje claro si la API no responde, `requirements.txt` con versiones fijas	3
6. Ficha y defensa (20)	Ficha completa: los 6 puntos de la sección 11	8
	Demostración en vivo	4
	Respuestas en la defensa sobre su propio código y decisiones	8
Extra	Red neuronal de Keras con curva de pérdida, early stopping y comparación explicada	+5

Descuentos:

- −10 si el notebook no corre con *Reiniciar y ejecutar todo* (salvo la celda que pide la contraseña).
- −5 por cada hito no entregado a tiempo.
- Una cifra en sus respuestas que no sale de sus propias salidas se considera no respondida.

13. Hitos, entrega y reglas

Un **hito** es una entrega parcial con fecha. Los hitos no tienen nota propia: sirven para recibir retroalimentación a tiempo. No entregarlos descuenta 5 puntos cada uno.

Entrega	Qué incluye	Fecha
Hito 1	Partes 1 y 2: notebook, `preparacion.py` y `modelo_riesgo.joblib`	[completar]
Hito 2	Parte 3 y la API desplegada (URL)	[completar]
Entrega final	Todo, incluidas las partes 4, 5 y 6	[completar]
Defensas	8 minutos por pareja	[completar]

Qué subir en la entrega final

Un solo `.zip` llamado `Apellido1_Apellido2_Proyecto_Final.zip`:

```
Proyecto_Final.ipynb          ← la plantilla, ejecutada completa
preparacion.py
modelo_riesgo.joblib
pronostico_junio.csv          ← solo si eligieron 3A
sentimiento_muestra.csv       ← o el nombre que usen en la Parte 4
ficha_modelo.md
api_url.txt                   ← una línea con la URL de Render
app_url.txt                   ← una línea con la URL de Streamlit
api/                          ← main.py, requirements.txt y lo que necesite
app/                          ← streamlit_app.py, requirements.txt y datos
```

Reglas

- Parejas. Un solo integrante sube el archivo; los dos nombres van en el notebook.
- Usen la plantilla `Plantilla_Proyecto_Final.ipynb` y no borren sus encabezados. Pueden agregar todas las celdas que necesiten.
- Los datos se leen desde la base de datos del curso, no desde archivos descargados de otro lado.
- **Uso de IA:** está permitido. Declaren en la sección final de la plantilla qué herramientas usaron y para qué. Todo lo entregado se defiende en vivo: si no pueden explicar una parte,

esa parte no cuenta.

- La API y la app deben seguir publicadas hasta que se publiquen las notas.

14. Preguntas frecuentes

¿Podemos agregar variables que no están en el contrato?
Sí, si se calculan dentro de su pipeline a partir de las columnas del contrato (en `preparacion.py`). No pueden pedir columnas nuevas como entrada.

¿Podemos usar `order_delivered_customer_date`? ¿No es del futuro?
Sí, se puede: el modelo predice en el momento de la entrega, así que esa fecha ya se conoce. Lo que no existe todavía es la reseña.

¿Por qué precisión en el 10% y no exactitud (accuracy)?
Porque solo 1 de cada 7 u 8 pedidos tiene mala reseña: un modelo que dice "nunca" acierta más del 85% y no sirve para nada. Lo que le importa al negocio es acertar en los pocos clientes que puede llamar.

Mi `.joblib` no carga en Render.
Casi siempre son versiones distintas: copien en `requirements.txt` exactamente las que imprime la plantilla. Si el error dice que no encuentra `crear_variables`, falta `preparacion.py` en la carpeta de la API.

El modelo de sentimiento tarda mucho.
Activen la GPU de Colab y pasen los textos en lote (`batch_size=32`). Si un comentario es muy largo, usen `truncation=True`.

¿Tenemos que hacer las dos opciones de la Parte 3?
No, una. Si hacen las dos, se califica la que indiquen como elegida.

¿Qué tan alto debe salir el desempeño?
No hay una cifra exacta que alcanzar: la escala de la sección 6 premia superar claramente a la regla del retraso. Concéntrense en una validación honesta; un modelo con fuga de datos se ve mejor en el notebook y peor con los pedidos ocultos.